Cloud-Based License Plate Recognition System

Technologies Used:

1 **Cloud Computing (AWS)**

2 **Amazon S3**

3 **Amazon ECS / Fargate**

4 **Docker**

5 **Python**

6 **OpenCV**

7 **OCR (Optical Character Recognition)**

8 **REST API**

Usage:

1 Automatic vehicle license plate detection and recognition.

2 Traffic monitoring and vehicle tracking.

3 Smart parking entry and exit management.

4 Security and surveillance system integration.

Step 1: Set Up the Watchlist Database (Amazon DynamoDB)

1 Open the **Amazon DynamoDB Console** and click **Create table**.

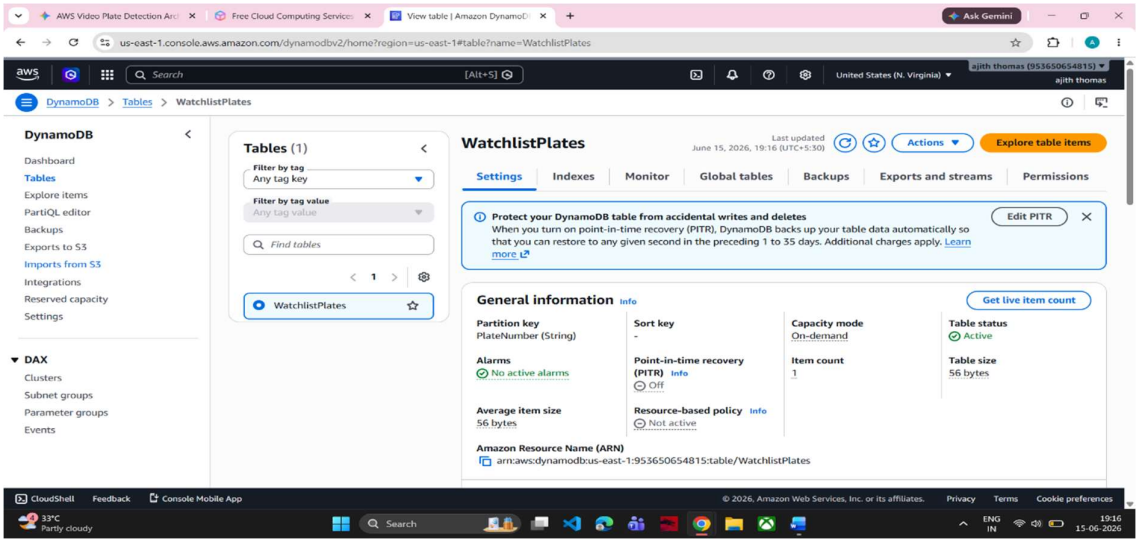
2 **Table name:** “WatchlistPlates”

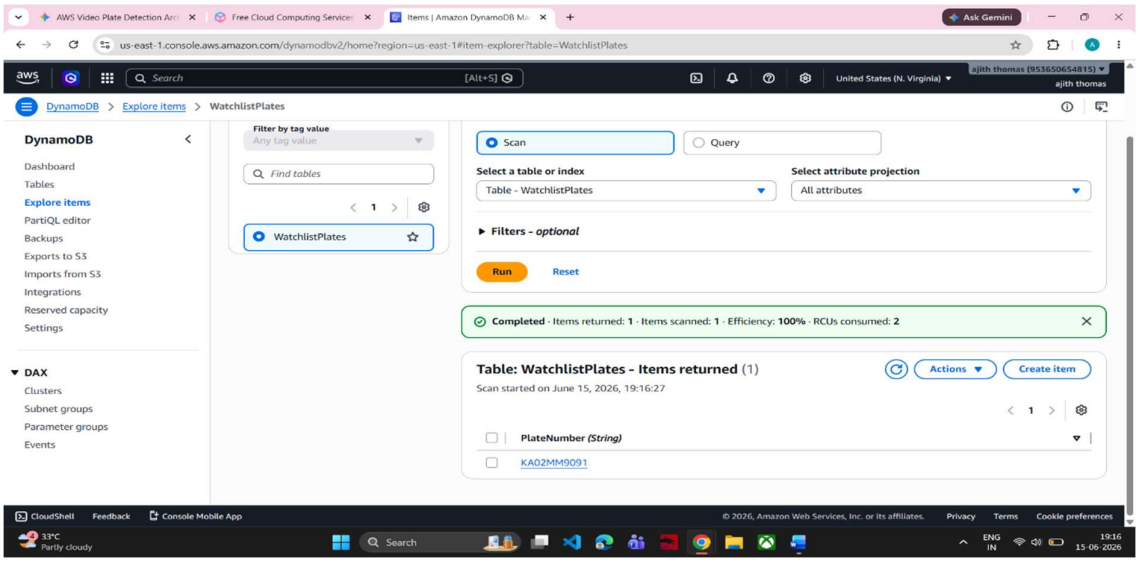
3 **Partition key:** “PlateNumber” (Type: **String**).

4 Leave all other settings as default and click **Create table**.

5 Once active, click on the table, select **Actions > Explore items**, and click **Create item**.

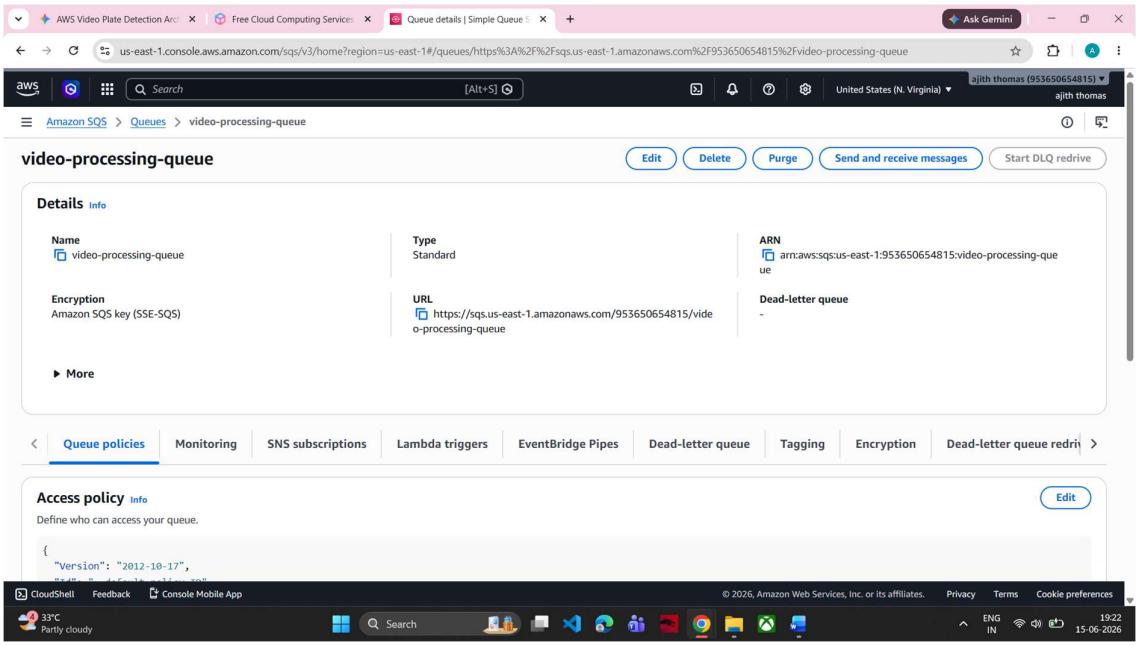
6 Enter a test plate number (e.g., KL20MM3454) so you can test the system later.





Step 2: Set Up the Messaging Queue (Amazon SQS)

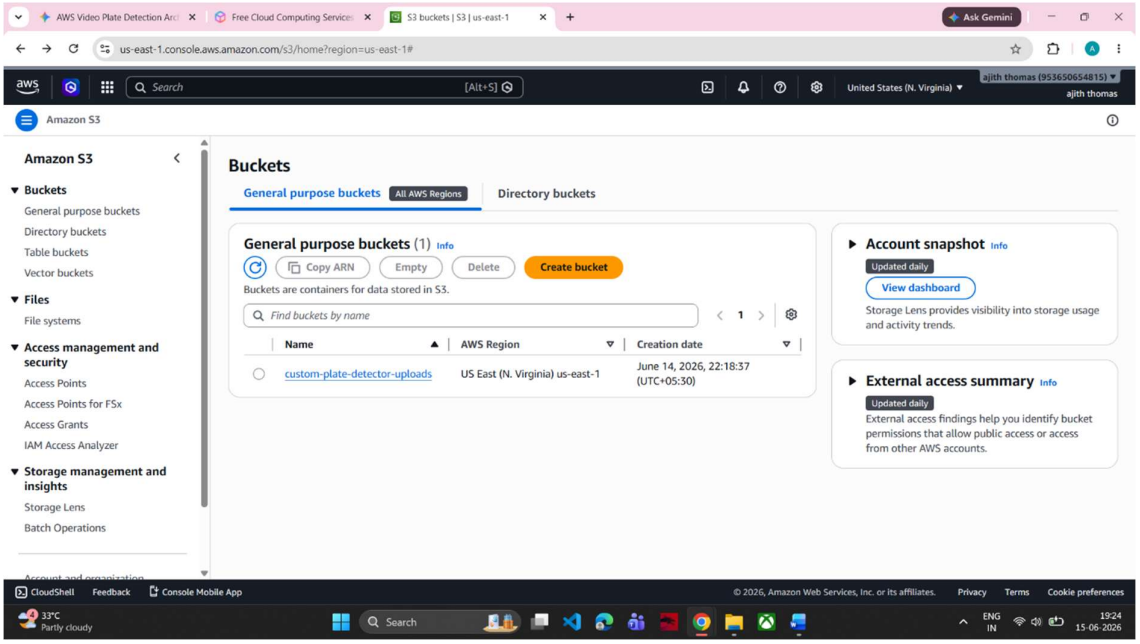
1. Open the **Amazon SQS Console** and click **Create queue**.
2. Select **Standard** queue.
3. **Name:** video-processing-queue.
4. Scroll to the bottom and click **Create queue**.
5. **CRITICAL:** Copy the **URL** of this queue (it will look like https://sqs.us-east-1.amazonaws.com/123456789012/video-processing-queue). You will need it in your code.



Step 3: Set Up the Video Storage (Amazon S3)

This is where users will upload their MP4 videos.

1. Open the **Amazon S3 Console** and click **Create bucket**.
2. **Bucket name:** Choose something unique (e.g., custom-plate-detector-uploads-2026).
3. Keep the default security settings ("Block all public access" should be checked) and click **Create bucket**.



Step 4: Create the S3-to-SQS Link (AWS Lambda)

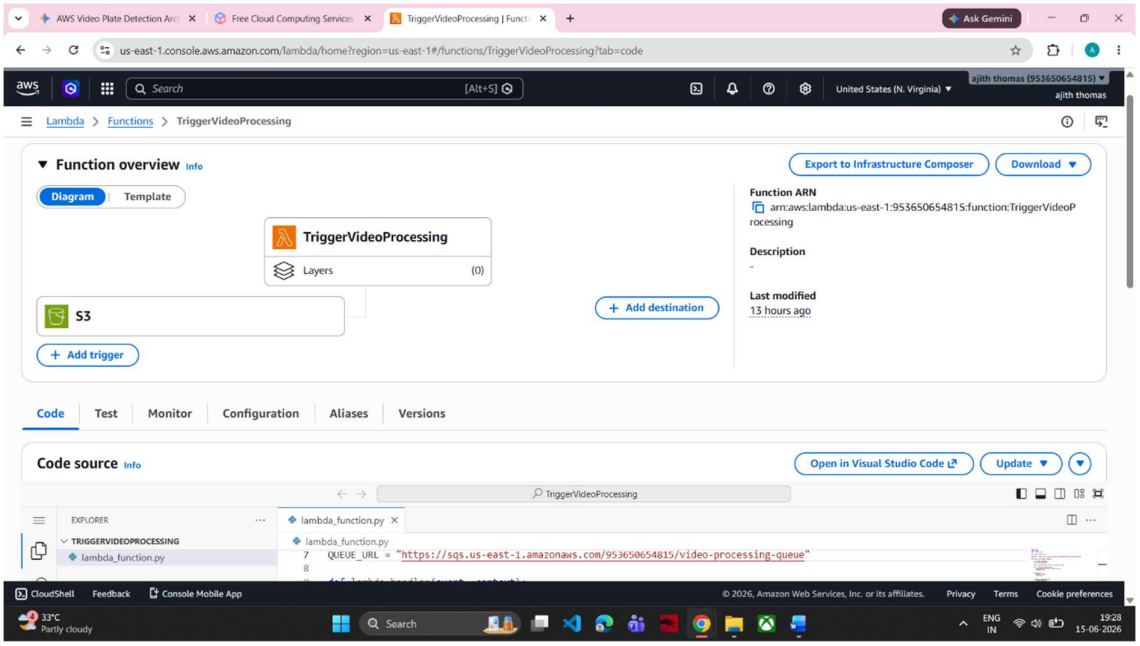
1. Open the **AWS Lambda Console** and click **Create function**.
2. **Name:** TriggerVideoProcessing | **Runtime:** Python 3.12.
3. Change the default execution role to create a new role with basic Lambda permissions.
4. Once created, go to the **Configuration** tab > **Permissions** and click on your **Role name** to open IAM.
5. In IAM, click **Add permissions > Attach policies**, and add **AmazonSQSFullAccess**. (This allows Lambda to send messages to your queue).
6. Go back to your Lambda function, clear the code editor, and paste this:

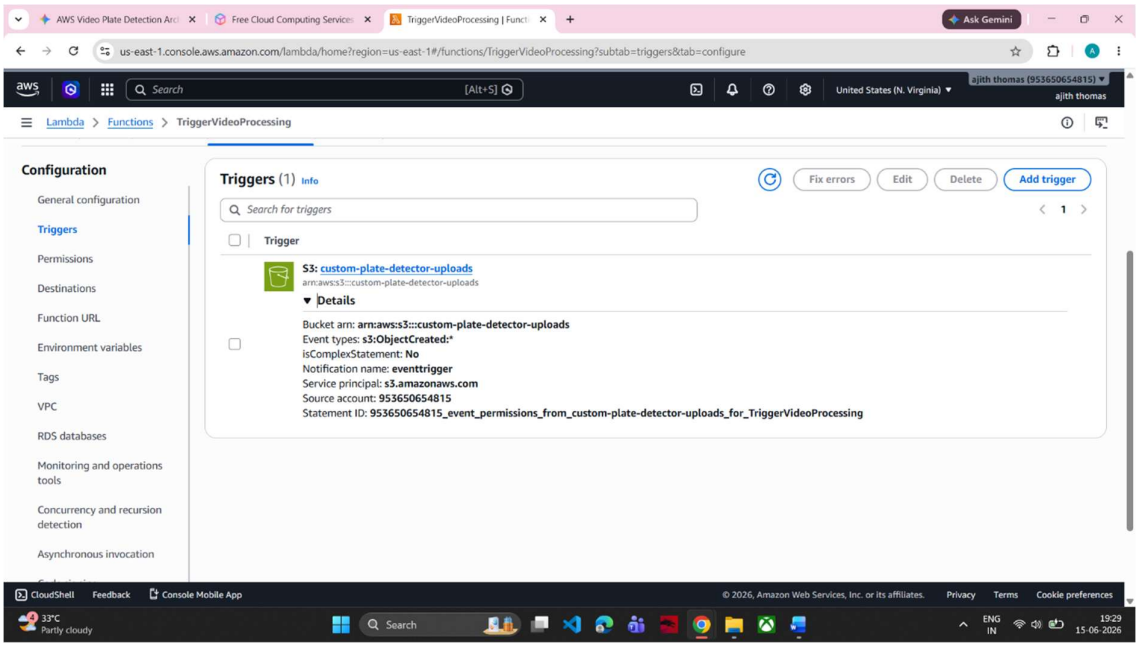
```
import json
import boto3
import urllib.parse
sqs = boto3.client('sqs')
QUEUE_URL = 'https://sqs.YOUR_REGION.amazonaws.com/YOUR_ACCOUNT_ID/video-processing-queue'
def lambda_handler(event, context):
    bucket = event['Records'][0]['s3']['bucket']['name']
    key = urllib.parse.unquote_plus(event['Records'][0]['s3']['object']['key'], encoding='utf-8')
    message_body = {
        'bucket': bucket,
        'key': key
    }
    sqs.send_message(
        QueueUrl=QUEUE_URL,
        MessageBody=json.dumps(message_body)
    )

    return {'statusCode': 200, 'body': json.dumps('Video queued successfully!')}
```

7 Click **Deploy**.

8 Scroll to the top of the Lambda page, click **Add trigger**, select **S3**, choose your bucket, leave event type as "All object create events", and click **Add**.

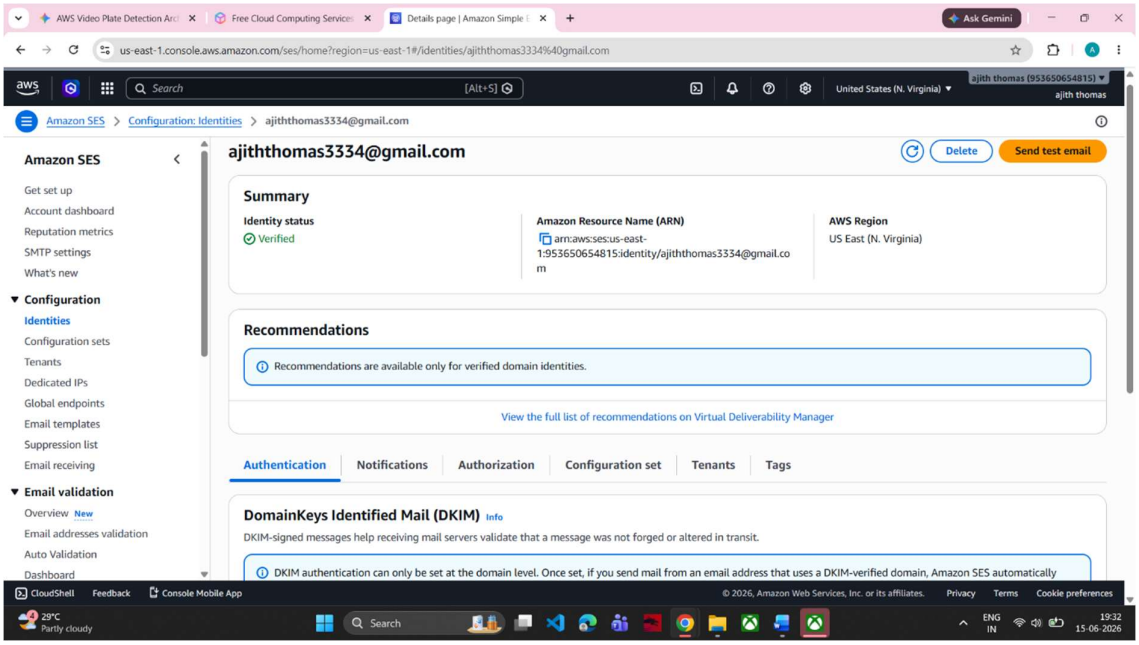




Step 5: Configure the Email Service (Amazon SES)

Before your code can send emails, AWS requires you to verify your email identity to prevent spam.

1. Open the Amazon SES Console.
2. Under Configuration, click Verified identities > Create identity.
3. Select Email address, type your email address, and click Create identity.
4. Check your personal email inbox, open the verification link sent by AWS, and verify it.



Step 6: Write the OpenCV + EasyOCR Worker Script

This Python script runs indefinitely. It continuously checks SQS for messages, downloads the video, processes frames with OpenCV and EasyOCR, and checks DynamoDB.

Create a folder on your local computer and save this file inside it as processor.py:

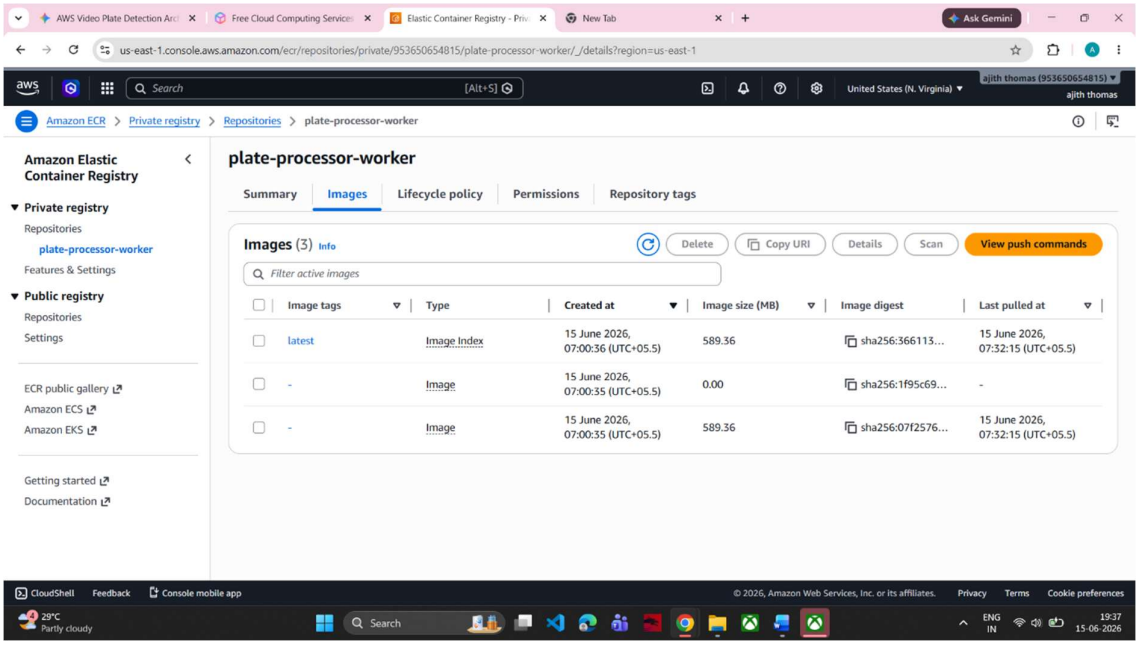
Step 7: Containerize with Docker

Because EasyOCR uses complex machine learning binaries (PyTorch), we package it into a Docker image so AWS can run it anywhere seamlessly.

Step 8: Push Image to Amazon ECR & Deploy to Fargate

A. Push Image to ECR

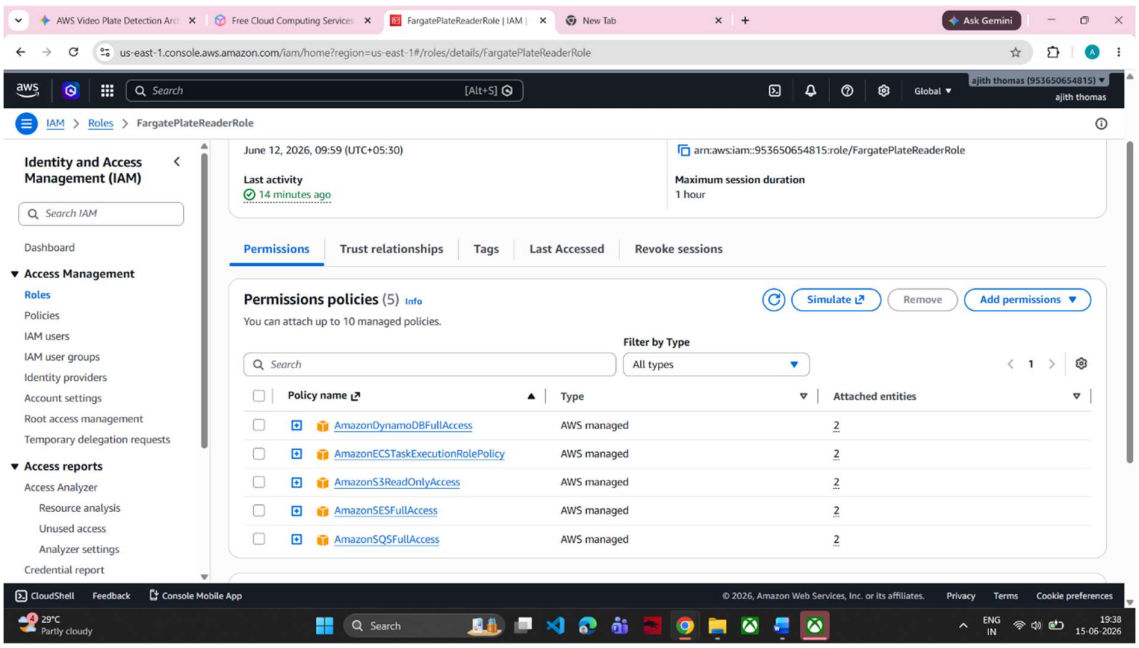
1. Go to the **Amazon ECR Console** and click **Create repository**. Name it plate-processor-worker.
2. Click into the repository and click **View push commands** in the top right.
3. Open your terminal in your local project folder and execute those 4 commands step-by-step to build and push your Docker container to AWS.
4. aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 953650654815.dkr.ecr.us-east-1.amazonaws.com
5. docker build -t plate-processor-worker .
6. docker tag plate-processor-worker:latest 953650654815.dkr.ecr.us-east-1.amazonaws.com/plate-processor-worker:latest
7. docker push 953650654815.dkr.ecr.us-east-1.amazonaws.com/plate-processor-worker:latest
8. Copy your **Image URI** when finished



B. Setup IAM Security Role for Fargate

Your container needs rights to talk to SQS, S3, DynamoDB, and SES.

1. Open the **IAM Console**, click **Roles > Create role**.
2. Select **AWS Service**, pick **Elastic Container Service**, and then select **ECR Task** as the use case.
3. Attach policies: AmazonSQSFullAccess, AmazonS3ReadOnlyAccess, AmazonDynamoDBFullAccess, and AmazonSESFullAccess. Name the role FargatePlateReaderRole.

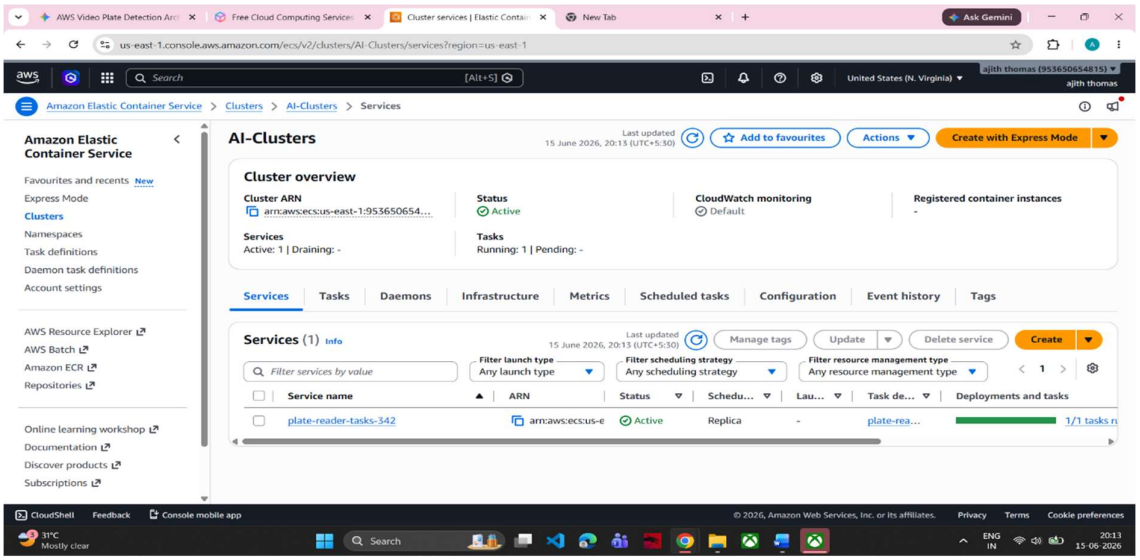


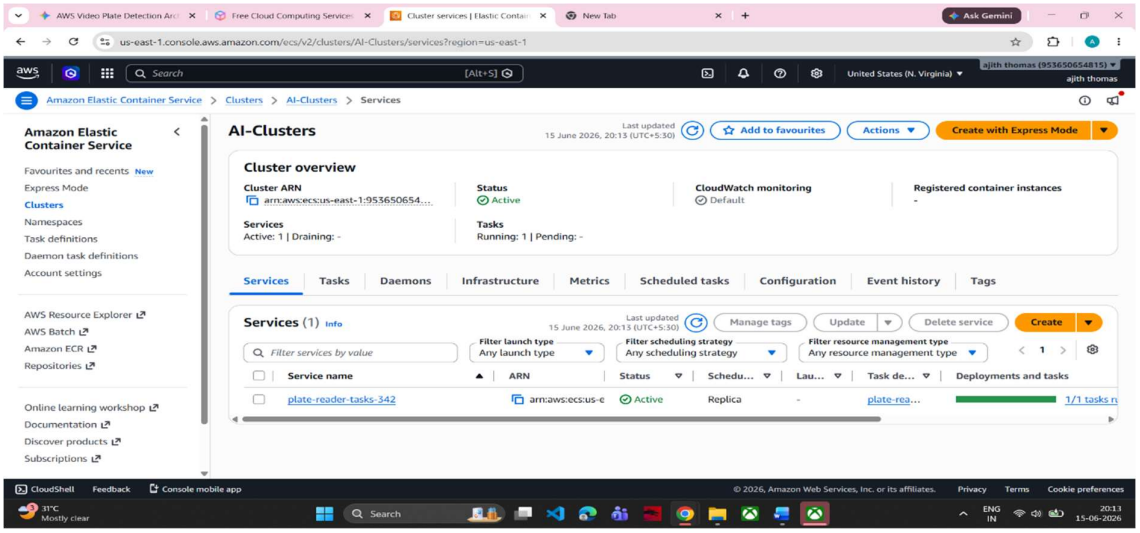
C. Deploy on AWS Fargate (ECS)

1. Go to **Amazon ECS Console**. Click **Clusters > Create cluster**. Select **AWS Fargate (serverless)** and name it AI-Cluster.
2. Go to **Task Definitions > Create new Task Definition**.
    - **Family Name:** plate-reader-task
    - **Infrastructure:** AWS Fargate.
    - **Task Role:** Choose the FargatePlateReaderRole you made in Step 8B.

- **Container Details:** Give it a name (worker), paste the **ECR Image URI** from step 8A.
- Set allocation sizing (at least **1 vCPU** and **3 GB Memory** to give PyTorch and EasyOCR enough room to breathe). Click **Create**.

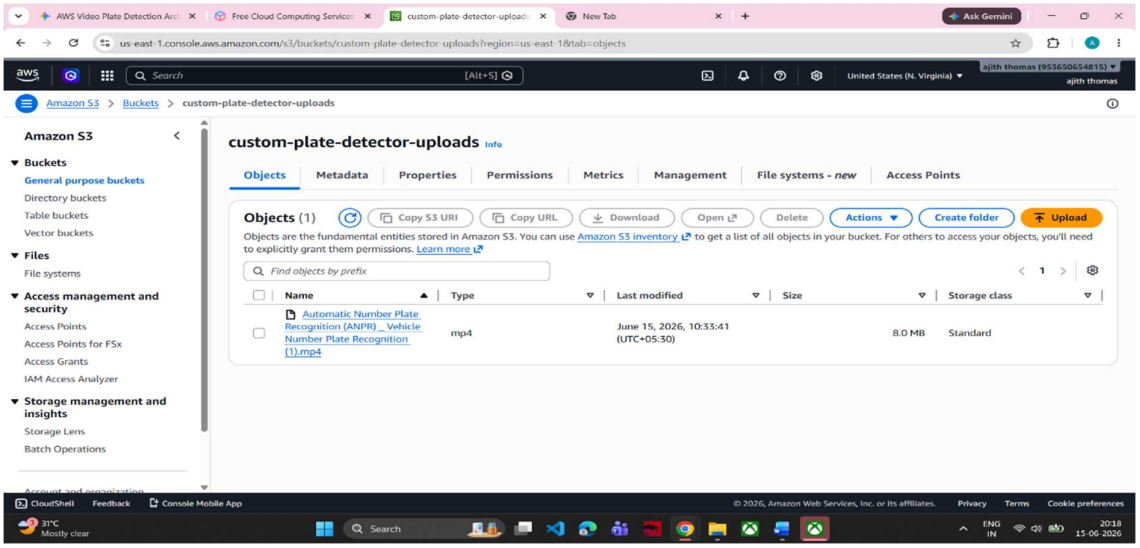
3. Go back to your AI-Cluster, click the **Services** tab, and click **Create**.
   - Launch Type: **Fargate**.
   - Task Definition Family: plate-reader-task.
   - Desired Tasks: **1** (This keeps exactly one copy of your script running 24/7 scanning for videos). Click **Create**.

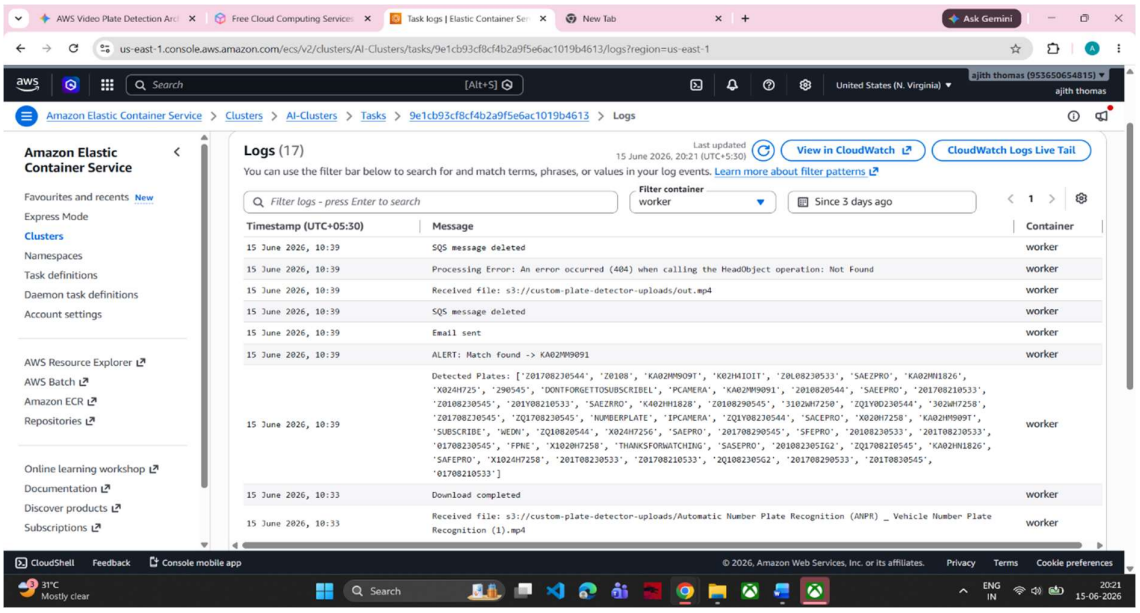




Step 9: Test Your Creation!

1. Open your S3 bucket.
2. Upload a video file containing a clear license plate matching the value you typed into DynamoDB in Step 1 (e.g., XYZ123).
3. Your S3 bucket triggers Lambda $\rightarrow$ Lambda sends details to SQS $\rightarrow$ Your Fargate container notices the queue item, downloads it, fires up EasyOCR, spots the match, and drops a security notification straight into your email inbox!





Step 9: Output